Лекция 4. ООП. Класс

1. ООП
   - Инкапсуляция
   - Наследование
   - Полиморфизм
   - Абстракция
2. Класс
   - Спецификаторы доступа
   - Отличие класса и структуры
   - Пустой класс
   - Поля класса
   - Инициализация полей значением по умолчанию (*C++11*)
   - Методы
   - Определение методов вне класса
   - Указатель на себя `this`
   - CV-квалификация методов
   - Ключевое слово `mutable`
   - Конструктор и специальные методы
   - Список инициализации полей класса
   - Параметризованный конструктор
   - Конструктор по умолчанию
   - Конструктор копирования
   - Копирующий оператор присваивания
   - Деструктор
   - Конструктор преобразования
   - Ключевое слово `explicit`
   - Конструктор от `std::initializer_list` (*C++11*)
   - Делегирующий конструктор (*C++11*)
   - Ключевое слово `default` (*C++11*)
   - Ключевое слово `delete` (*C++11*)
   - Оператор преобразования
   - Перегрузка операторов внутри класса
   - Перегрузка операторов вне класса
   - Ключевое слово `friend`
   - Оператор трехстороннего сравнения (*C++20*)
   - Указатели на члены класса. Операторы `.*` и `->*`

ООП

Объектно-ориентированное программирование - парадигма программирования, которая основывается на представлении в коде программы различных объектов, взаимодействующих друг с другом.

Класс - пользовательский тип данных, шаблон (макет) для создания объектов и описания их характеристик, функций.

Объект - экземпляр класса. Объект включает данные (поля) и методы (функции). Что позволяет хранить характеристики объекта, изменять их и взаимодействовать с другими объектами.

Основные принципы ООП:

- Инкапсуляция
- Наследование
- Полиморфизм
- Абстракция

Инкапсуляция

Инкапсуляция - объединение данных и методов для работы с данными внутри класса. Сокрытие деталей реализации класса.

```
class Budget {
public:
    void IncreaseBalance(double value) {
        budget_ += value;
    }
private:
    double budget_ = 0.0;
};
```

Наследование

Наследование - механизм создания новых классов на основе существующих. Позволяет строить иерархию классов и переиспользовать код классов родителей внутри классов наследников.

```
class Animal { /* common data */ };
class Cat : public Animal {};
class Dog : public Animal {};
```

Полиморфизм

Полиморфизм - возможность различного поведения сущностей C++.

Виды полиморфизма:

- статический (на этапе компиляции, шаблоны, перегрузка функций)
- динамический (во время выполнения программы, виртуальные методы)

```
class Shape {
public:
    virtual ~Shape() = default;
    virtual void Draw() = 0;
};
class Circle : public Shape {
public:
    void Draw() override { /* рисуем круг */ }
};
```

- `override` (C++11) - проверка компилятором, что метод действительно переопределяет виртуальный метод базового класса

Абстракция

Абстракция - упрощение сложных вещей через выделение основных характеристик. Например, пользователю `Shape` достаточно знать, что фигуру можно нарисовать, а как именно рисуется круг, ему знать не нужно.

Класс

Класс - пользовательский тип данных, который объединяет в себе данные (поля класса) и функции для работы с данными (методы класса), представляет собой макет для создания объектов (экземпляров) данного типа.

Синтаксис: `class <name> {<class_body>};`

- `<name>` - имя класса, пользовательского типа данных
- `<class_body>` - тело класса, включающее поля, методы, конструкторы и деструктор

Спецификаторы доступа

Для ограничения видимости полей и методов внутри класса используются спецификаторы доступа. Все члены класса, объявленные после спецификатора (до следующего спецификатора или до конца класса), имеют соответствующий доступ:

- `public` - публичный доступ, поле или метод класса доступны извне

- `protected` - защищенный доступ, поля и методы доступны наследникам класса и внутри класса
- `private` - приватный доступ, поля и методы доступны только внутри класса

Синтаксис внутри класса или структуры: `<specifier>:`

Указывать спецификаторы доступа можно произвольное число раз.

```
class User {
public:
    /* some data and functions for everyone */
protected:
    /* some data and functions for children classes */
private:
    /* some data and functions inside class */
};
```

- как правило, спецификаторы располагают без отступов

Приватные поля и методы будут недоступны снаружи, то есть **НЕЛЬЗЯ** к ним обратиться или вызвать через экземпляр класса, используя операторы `.`, `->`. Доступ к `private` и `protected` членам есть также у друзей класса (`friend` , о них в конце лекции).

Всё содержимое класса по умолчанию имеет спецификатор доступа `private`, несмотря на это часто принято явно указывать данный спецификатор, даже при определении полей класса в самом начале тела класса.

Отличие класса и структуры

Структура `struct` и класс `class` имеют одинаковые возможности в C++.

Отличие заключается в том, что содержимое структуры по умолчанию имеет публичный доступ `public`, а содержимое класса приватный `private`.

Несмотря на одинаковые возможности, принято разделять структуру и класс семантически. Так, структуру используют только с публичными полями (простой набор данных), а класс - с приватными. Создавать классы и структуры со смешанным типом полей **НЕ** рекомендуется, так как это может быть не очевидно и не понятно программистам, читающим код.

Также структуру используют для взаимодействия с legacy-кодом на языке C.

Начиная с *C++20*, класс или структура с любым объявленным пользователем конструктором **НЕ** является агрегатом, и инициализация в фигурных скобках `{1, 2}` вызывает конструктор, а не агрегатную инициализацию полей.

Пустой класс

Пустой класс имеет размер 1 байт, поскольку объект такого класса можно создать и необходимо иметь адрес данного объекта. Чтобы иметь адрес, необходимо что-то положить в память по определенному адресу.

Чаще используется пустая структура. Такая структура может понадобиться в качестве именованного тега. Пока будем просто считать, что иногда надо.

Поля класса

Поля класса представляют собой внутренние переменные произвольного типа. К полям класса внутри класса можно обращаться по имени. В качестве поля можно использовать указатели и ссылки.

В случае ссылок необходимо их инициализировать при создании объекта. Например, можно проинициализировать ссылкой на объект (адресом объекта) из глобальной области видимости. А еще это можно сделать в списке инициализации при конструировании объекта.

Объект с полем-ссылкой или константным полем можно скопировать, но **НЕЛЬЗЯ** присвоить: компилятор не сгенерирует для него оператор присваивания. Поэтому вместо ссылки в классе обычно хранят указатель.

Существуют разные стили кода к именованию полей класса. Часто встречается:

- `m_<name>` - добавляют `m_` в качестве префикса к переменной (`m` - `member`)
- `<name>_` - добавляют `_` в качестве постфикса к переменной

```
// inside class
int m_value;
int value_;
```

Поля класса хранятся в классе и инициализируются в порядке их объявления.

Поля уникальны для каждого экземпляра класса.

Инициализация полей значением по умолчанию (*C++11*)

Аналогично структурам рекомендуется всегда инициализировать поля внутри класса.

```
class Time {
private:
    int hour_ = 0;
    int minute_{0}; // uniform
};
```

Иначе в полях класса также может лежать мусор. Чтение неинициализированного поля - **UB**, а начиная с *C++26* - ошибочное поведение (*erroneous behavior*): значение **НЕ** определено это

уже **НЕ UB**, и компилятор вправе диагностировать ошибку.

Методы

Внутри класса можно определять функции, которые могут работать с полями класса, в том числе закрытыми. Данные функции называются методы.

Синтаксис аналогичен определению обычных функций.

Публичный метод можно вызвать через операторы `.` для экземпляра и `->` для указателя.

Приватные методы можно вызывать только внутри класса.

Можно вызывать методы в списках инициализации. Например, метод, который будет контролировать допустимость значения или выполнять дополнительные преобразования. Такой метод **НЕ** должен читать поля, которые еще **НЕ** проинициализированы.

Метод, определенный внутри класса, неявно является `inline`, поэтому класс с такими методами можно определять в заголовочном файле.

Определение методов вне класса

Методы можно объявить внутри класса, а определить снаружи класса. Содержимое класса имеет свою область видимости. Для определения снаружи класса перед именем конструктора, метода, оператора используется имя класса и оператор разрешения области видимости `::`

```
class Time {
public:
    Time();
    Time(int hours, int minutes, int seconds = 0);
    int GetHours();
    void SetHours(int hours);
private:
    int hours_ = 0;
    int minutes_ = 0;
    int seconds_ = 0;
};

Time::Time() = default;
Time::Time(int hours, int minutes, int seconds)
    : hours_(hours), minutes_(minutes), seconds_(seconds) {}

int Time::GetHours() { return hours_; }
void Time::SetHours(int hours) { hours_ = hours; }
```

- определить методы можно в другом файле (как правило, в файле реализации `.cpp`)

Значения аргументов по умолчанию указываются только при объявлении, один раз. При определении вне класса повторить их нельзя, даже то же самое значение.

Если определение метода вне класса находится в заголовочном файле, его необходимо пометить `inline`, иначе при включении заголовка в несколько `.cpp` файлов будет ошибка линковки (нарушение ODR).

Указатель на себя `this`

Внутри класса, в методах, в том числе конструкторах, можно получить указатель на себя (объект класса, для которого вызван метод) с помощью ключевого слова `this`.

Можно использовать `this`, как в качестве значения по умолчанию, так и в списке инициализации.

```
class Self {
public:
    Self* GetPtr() { return self_; }
    Self& GetRef() { return *this; }
private:
    Self* self_ = this;
};
```

- **ВАЖНО**, что при копировании `Self b = a;`, поле `b.self_` будет указывать на `a`, поскольку сгенерированный конструктор копирования, скопирует содержимое поля как есть, то есть просто скопирует адрес (часто это **НЕ** корректное поведение)

Можно считать, что указатель на себя передается первым неявным аргументом в конструкторы, методы и операторы класса.

Через указатель можно явно обращаться к полям класса, но, как правило, так **НЕ** делают.

```
// inside methods
this->self_;
```

CV-квалификация методов

Методы могут иметь CV-квалификацию. Методы, которые не изменяют полей класса, а только предоставляют информацию о них, следует помечать квалификатором `const` после сигнатуры и перед телом метода:

```
class Size {
public:
    size_t GetSize() const { return size_; }
    void AddSize(size_t size) { size_ += size; }
private:
```

```
    size_t size_ = 0;
};
```

Методы, помеченные квалификатором `const`, можно вызывать у константных объектов класса. Компилятор отслеживает, что в данном методе нет изменений полей класса. Внутри `const` метода указатель `this` имеет тип `const Size*`

Не изменяет поля класса и может быть вызван для константного объекта:

```
int Class::Foo() const;
```

Может изменять поля класса и может быть вызван для `volatile` объекта:

```
int Class::Foo() volatile;
```

Может быть вызван как для `const`, так и для `volatile` объекта, так и для `const volatile` объекта:

```
int Class::Foo() const volatile;
```

Методы можно перегрузить, если они отличаются только квалификатором `const` (а также `volatile`). Например, `operator[]` у контейнеров:

```
class Array {
public:
    int& operator[](size_t id) { return data_[id]; }
    const int& operator[](size_t id) const { return data_[id]; }
private:
    int data_[10] = {};
};
```

- для константного объекта вызовется вторая версия, и изменить элемент **НЕ** получится

Ключевое слово `mutable`

Спецификатор типа `mutable` разрешает изменять поле класса, объявленное с ним, даже в константных методах и для константных объектов.

Например, это может быть поле, представляющее собой счетчик операций и необходимо его изменять даже в константном методе. Также `mutable` подходит для кэширования значения.

```
class Triangle {
public:
    explicit Triangle(double a, double b, double c) : sides_{a, b, c} {}
    double Perimeter() const {
        if (!cached_) {
```

```
            perimeter_ = 0;
            for (double s : sides_) perimeter_ += s;
            cached_ = true;
        }
        return perimeter_;
    }
private:
    std::vector<double> sides_;
    mutable bool cached_ = false;
    mutable double perimeter_ = 0;
};

const Triangle t(3, 4, 5);
t.Perimeter();  // считает, запоминает в кэше и возвращает 12
t.Perimeter();  // возвращает из кэша
```

Также `mutable` используется в лямбда-выражениях, но там у него другой смысл - разрешает изменять захваченные по значению копии.

Конструктор и специальные методы

Конструктор - это особый метод класса, который используется для конструирования объекта.

Синтаксис: `<class_name>(<arguments>) {<ctor_body>}`

- `<class_name>` - имя конструктора должно совпадать с именем класса
- `<arguments>` - аргументы конструктора
- `<ctor_body>` - тело конструктора

В зависимости от аргументов выделяют различные виды конструкторов:

- Параметризованный конструктор
- Конструктор по умолчанию
- Конструктор копирования
- Конструктор перемещения (*C++11*)
- Конструктор преобразования
- Конструктор от `std::initializer_list` (*C++11*)
- Делегирующий конструктор (*C++11*)

Также выделяют специальные методы класса (*special member functions*):

- Конструктор по умолчанию (*default constructor*)
- Деструктор (*destructor*)
- Конструктор копирования (*copy constructor*)
- Оператор присваивания копированием (*copy assignment operator*)
- Конструктор перемещения (*move constructor*, *C++11*)
- Оператор присваивания перемещением (*move assignment operator*, *C++11*)

Компилятор может самостоятельно сгенерировать шесть специальных методов. Объявленный пользователем конструктор отменяет только генерацию конструктора по умолчанию, остальные правила разберем в следующей лекции.

Сгенерированные методы работают поэлементно: копируют, перемещают и разрушают каждое поле. Если поле является указателем, это означает копирование адреса, а **НЕ** данных.

Важно понимать, что при входе в тело конструктора все поля уже проинициализированы и в теле может происходить только присваивание новых значений полям класса. Следовательно, в теле уже нельзя изменить константное поле или инициализировать ссылку.

Кроме значения по умолчанию, проинициализировать константу и ссылку можно в списке инициализации.

Список инициализации полей класса

Список инициализации полей (*member initializer list*) позволяет инициализировать поля в момент создания объекта. В списке инициализации доступны аргументы конструктора и имена полей класса. Список инициализации указывается между сигнатурой и телом конструктора, и выглядит как перечисление после символа `:` через запятую полей класса и внутри `()` или `{}` их инициализирующих значений.

Синтаксис: `<class_name>(<arguments>) : <mem_init_list> {<ctor_body>}`

- `<mem_init_list>` - список инициализации = `<field>(<value>), <field>{<value>}`

```
class InitList {
public:
    InitList(int val) : a_(val), b_(val), c_(val) {}
public:
    int a_;
    int b_;
    int c_;
};
```

Причем имена полей класса и имена параметров могут полностью совпадать, конфликта имен не будет, поскольку компилятор понимает, что нужно инициализировать поля.

```
class InitSameName {
public:
    InitSameName(int a, int b, int c) : a(a), b(b), c(c) {}
public:
    int a;
    int b;
    int c;
};
```

- внутри тела конструктора имя `a` будет означать параметр, а к полю придется обращаться через `this->a`

Также следует отметить, что в качестве инициализирующего значения может использоваться не только переменная или константа, но также выражение (*expression*) и результат вызова функции.

ВАЖНО, что инициализация происходит в порядке полей класса, и не зависит от порядка в списке инициализации. Поэтому важно самостоятельно отслеживать правильный порядок инициализации.

```
class BadOrderInit {
public:
    BadOrderInit(int val) : c(val), b(c + 1), a(10) {}
public:
    int a;
    int b;
    int c;
};
```

- `c` используется неинициализированной при инициализации `b` (**UB**)

Компиляторы предупреждают о таком порядке (`-Wreorder` входит в `-Wall`):

```
GCC: error: 'BadOrderInit::c' will be initialized after [-Werror=reorder]
Clang: error: initializer order does not match the declaration order [-Werror,-
Wreorder-ctor]
Clang: error: field 'c' is uninitialized when used here [-Werror,-
Wuninitialized]
```

Если поля класса объявлены со значением по умолчанию, то они будут проигнорированы для полей в списке инициализации.

```
class BadOrderInitDefault {
public:
    BadOrderInitDefault(int val) : c(val), b(c + 1), a(10) {}
public:
    int a = 7;
    int b = 7;
    int c = 7;
};
```

- значение `7` будет проигнорировано, по-прежнему **UB**

Списки инициализации могут быть неполными. Тогда недостающие поля будут сконструированы со значениями по умолчанию, а при их отсутствии остаются неинициализированными (мусор).

```
class BadOrderInitPartial {
public:
    BadOrderInitPartial(int val) : c(val) {}
public:
    int a; int b = 7; int c = 7;
};
```

- в поле `b` будет значение `7`, в `a` будет мусор

Список инициализации позволяет сконструировать константное поле и поле-ссылку извне:

```
class RefConst {
public:
    RefConst(int value, int& ref, const double& cref)
        : id_(value), ref_(ref), const_ref_(cref) {}
private:
    const int id_;
    int& ref_;
    const double& const_ref_;
};
```

ВАЖНО. Поле-ссылка `const_ref_` **НЕ** продлевает жизнь временного объекта. Таким образом, `RefConst r(1, x, 3.14)` оставит поле `const_ref_` висячей ссылкой, обращение к полю - **UB**. Компиляторы об этом **НЕ** предупредят.

Параметризованный конструктор

Конструктор, который имеет параметры (аргументы), называют параметризованным конструктором (конструктором с параметрами). Аргументов может быть несколько и они могут иметь значения по умолчанию. Конструкторы могут быть перегружены.

```
class Time {
public:
    Time(int hour, int minute, int second) :
        hour_(hour), minute_(minute), second_(second) {}
private:
    int hour_;
    int minute_;
    int second_;
};
```

Если конструктор имеет у всех аргументов значение по умолчанию, то такой конструктор является конструктором по умолчанию.

```
class Time {
public:
    Time(int hour = 0, int minute = 0, int second = 0)
```

```
        : hour_(hour), minute_(minute), second_(second) {}
private:
    int hour_;
    int minute_;
    int second_;
};
```

Для создания объекта класса необходим вызов конструктора. Синтаксис вызова конструктора:

```
Time t1(1, 1, 1);
Time t2{1, 1, 1};
Time t3 = {1, 1, 1};
Time t4 = Time{1, 1, 1};
Time t5 = Time(1, 1, 1);
```

Аналогично для всех его вариантов перегрузки.

Конструктор по умолчанию

Конструктор по умолчанию представляет собой конструктор без аргументов. Часто для простых классов конструктор имеет пустое тело. Удобно использовать значение по умолчанию для инициализации.

Синтаксис: `<class_name>() {<ctor_body>}`

Если в классе **НЕ** объявлен ни один конструктор, то компилятор самостоятельно сгенерирует данный конструктор.

```
class DefaultCtor {
public:
    DefaultCtor() {}
private:
    int value = 0;
};
```

Вызов конструктора:

```
DefaultCtor obj;
DefaultCtor obj2{};
DefaultCtor obj3 = {};
DefaultCtor obj4 = DefaultCtor{};
DefaultCtor obj5 = DefaultCtor();
```

- во всех этих случаях вызовется только конструктор по умолчанию один раз (для `obj4`, `obj5` гарантированно начиная с *C++17*)

ВАЖНО: пустые круглые скобки **НЕ** создают объект. Инструкция `DefaultCtor obj();` - это объявление функции `obj`, возвращающей `DefaultCtor` (*most vexing parse*). Компиляторы

предупреждают с опцией `-Wvexing-parse`.

Конструктор копирования

Конструктор копирования необходим для создания копии объекта из объекта того же типа. Представляет собой конструктор, принимающий в качестве аргументов константную ссылку того же типа, что и сам класс.

Принять объект по значению нельзя (ошибка компиляции): для передачи по значению самому конструктору копирования понадобилась бы копия.

Синтаксис: `<class_name>(const <class_name>& <other>) {<ctor_body>}`

```
class Time {
public:
    Time() {}
    Time(const Time& other) :
        hour_(other.hour_), minute_(other.minute_), second_(other.second_) {}
private:
    int hour_ = 0;
    int minute_ = 0;
    int second_ = 0;
};
```

- если убрать `Time() {}`, то `Time t;` не скомпилируется: объявленный конструктор копирования отменяет генерацию конструктора по умолчанию

Поля другого объекта того же класса доступны внутри методов класса, даже если они приватные.

Вызывается конструктор копирования:

- при передаче в функцию по значению
- при конструировании одного объекта из другого
- при возврате объекта по значению, только если копию нельзя опустить или заменить перемещением (подробнее в следующей лекции)

```
Time t;
Time t1 = t; // copy ctor
Time t2(t);  // copy ctor
Time t3{t};  // copy ctor
```

Копирующий оператор присваивания

Оператор присваивания необходим при присваивании одного созданного объекта другому. Если объект слева только создаётся, то он **НЕ** будет вызываться, а будет вызываться

конструктор копирования, даже если в инструкции есть `=`.

Как правило, оператор возвращает ссылку на себя (экземпляр текущего класса), что позволяет использовать цепочку из операторов `=`. Для этого необходимо вернуть из оператора разыменованный указатель на себя `return *this;`.

Синтаксис: `<class_name>& operator=(const <class_name>& <other>) {<body>}`

Поскольку язык не запрещает присвоить объект самому себе, как правило, в копирующем операторе присваивания выполняют проверку на самоприсваивание. Особенно это критично для классов, владеющих ресурсами (выделяющих память), что может привести к **UB**.

```
class Time {
public:
    Time() {}
    Time& operator=(const Time& other) {
        if (this == &other) {
            return *this;
        }
        hour_ = other.hour_;
        minute_ = other.minute_;
        second_ = other.second_;
        return *this;
    }
private:
    int hour_ = 0;
    int minute_ = 0;
    int second_ = 0;
};
```

Вызов оператора:

```
Time t1, t2, t3;
t1 = t2;      // copy assignment
t1 = t1;      // copy assignment
t1 = t2 = t3; // copy assignment
auto t4 = t1; // copy ctor (not a copy assignment!)
```

- Clang предупреждает о `t1 = t1;` (`-Wself-assign-overloaded` входит в `-Wall`)
- для `auto t4 = t1;` оба компилятора предупреждают (`-Wdeprecated-copy` входит в `-Wextra`): если пользователь написал оператор присваивания, то неявный конструктор копирования устарел, нужно написать оба.

Деструктор

Особый метод, вызываемый перед разрушением объекта, когда заканчивается время жизни объекта.

Синтаксис: `~<class_name>() {}`

Если деструктор не объявлен, компилятор сгенерирует его сам: он вызовет деструкторы полей. Писать деструктор нужно, когда класс вручную владеет ресурсом (память, файл, мьютекс, сокет). В этом случае в конструкторе ресурсы выделяются вручную, а в деструкторе освобождаются вручную. При владении ресурсом почти всегда нужны и другие специальные методы.

Деструкторы вызываются в обратном порядке по отношению к конструируемым объектам при выходе из области видимости. Последний сконструированный объект будет разрушен первым. Поля разрушаются в обратном порядке объявления после выхода из тела деструктора.

```
struct Member {
    explicit Member(std::string name) : name_(name) {
        std::cout << "ctor " << name_ << '\n';
    }
    ~Member() { std::cout << "dtor " << name_ << '\n'; }
    std::string name_;
};

struct Holder {
    Holder() { std::cout << "ctor Holder\n"; }
    ~Holder() { std::cout << "dtor Holder\n"; }
    Member first_{"first Holder"};
    Member second_{"second Holder"};
};

int main() {
    Member a{"Member a"};
    Holder h;
    Member b{"Member b"};
}
```

Вывод программы:

```
ctor Member a
ctor first Holder
ctor second Holder
ctor Holder
ctor Member b
dtor Member b
dtor Holder
dtor second Holder
dtor first Holder
dtor Member a
```

- поля конструируются в порядке объявления до входа в тело конструктора

Деструктор неявно `noexcept` (C++11), исключение из него завершит программу.

Конструктор преобразования

Конструктором преобразования называется конструктор, принимающий один аргумент другого произвольного типа. Данный конструктор разрешает неявное преобразование из указанного типа в тип класса.

```
class Integer {
private:
    int value;
public:
    Integer(int v) : value(v) {}
    Integer(char c) : value(static_cast<int>(c)) {}
};
```

Таким образом, если функция принимает пользовательский класс, а класс имеет конструктор преобразования от другого типа, то в функцию можно передать непосредственно этот другой тип, произойдет неявное преобразование с помощью соответствующего конструктора:

```
void DoSomething(Integer) {}

int main() {
    Integer i{3};
    int value = 5;
    char c = 'I';
    DoSomething(i);     // OK
    DoSomething(value); // OK
    DoSomething(5);     // OK
    DoSomething(c);     // OK
    DoSomething('i');   // OK
}
```

ВАЖНО понимать, что при наличии конструктора присваивания из другого типа, компилятор **НЕ** будет генерировать оператор присваивания из данного типа.

Отдельный оператор присваивания из другого типа писать **НЕ** обязательно: `i = 5;` создаст временный объект соответствующего типа через конструктор преобразования и вызовет обычный оператор присваивания для него.

Оператор присваивания из другого типа `operator=(int)` пишут, только чтобы избежать создания временного объекта.

Ключевое слово `explicit`

Ключевое слово `explicit` используется как спецификатор перед именем конструктора и позволяет запретить неявное преобразование и сообщает компилятору, что данный конструктор можно вызывать только явно.

```
class Integer {
public:
    Integer(int v) : value(v) {}
```

```
    Integer(char c) : value(static_cast<int>(c)) {}
    explicit Integer(double d) : value(static_cast<int>(d)) {}
private:
    int value = 0;
};
```

Неявно такой конструктор вызвать нельзя:

```
//Integer i2 = 3.14;        // compile error
Integer i3 = Integer{3.14}; // OK
Integer i4(3.14);           // OK
Integer i5{3.14};           // OK

void DoSomething(Integer) {}

int main() {
    double d = 3.14;
    //DoSomething(d);             // compile error
    //DoSomething(3.14);          // compile error
    DoSomething(Integer{3.14}); // OK
    DoSomething(Integer(3.14)); // OK
}
```

Рекомендуется помечать `explicit` все конструкторы с одним аргументом, если неявное преобразование не является естественным для типа.

Также спецификатор `explicit` можно использовать с оператором преобразования (*C++11*), об этом после знакомства с методами.

Конструктор от `std::initializer_list` (*C++11*)

В C++11 появился легковесный объект-обёртка над массивом - список инициализации `std::initializer_list`, который позволяет инициализировать класс набором элементов. Данный конструктор вызывается при перечислении элементов через запятую в фигурных скобках. Что вызывает неоднозначность при наличии других подходящих конструкторов какой конструктор вызывать.

Конструктор по умолчанию имеет приоритет перед конструктором от списка инициализации при пустых `{}`.

Список инициализации имеет приоритет перед параметризованными конструкторами при использовании `{}`.

```
class Vector {
public:
    Vector() {}
    Vector(size_t count);
    Vector(int a, int b);
    Vector(std::initializer_list<int> list);
```

```
private:
    std::vector<int> data;
};
```

Вызов конструкторов:

```
Vector v = {1, 2, 3, 4, 5}; // ctor std::initializer_list
Vector v2{1, 2, 3};         // ctor std::initializer_list
Vector v3(10);              // ctor Vector(size_t)
Vector v4{10};              // ctor std::initializer_list
Vector v5 = {10};           // ctor std::initializer_list
Vector v6(10, 20);          // ctor Vector(int a, int b)
Vector v7{10, 20};          // ctor std::initializer_list
Vector v8 = {10, 20};       // ctor std::initializer_list
Vector v9 = 10;             // ctor Vector(size_t) implicit cast
Vector v10;                 // default ctor
Vector v11{};               // default ctor
Vector v12 = {};            // default ctor
```

Приоритет списка инициализации сохраняется, даже если он приводит к ошибке:

```
size_t n = 10;
Vector v13(n);  // ctor Vector(size_t)
// Vector v14{n}; // compile error: narrowing size_t -> int
```

- во втором случае выбран конструктор от `std::initializer_list<int>`, а сужающее преобразование внутри `{}` запрещено, хотя `Vector(size_t)` подошёл бы идеально

Делегирующий конструктор (*C++11*)

Делегирующий конструктор - конструктор, который на месте списка инициализации использует другой конструктор данного класса. В таком случае можно указать только один целевой конструктор, другие инициализаторы полей в списке указать нельзя.

```
class Time {
public:
    Time(int hour, int minute, int second)
        : hour_(hour), minute_(minute), second_(second) {}
    Time(int hour) : Time(hour, 0, 0) {}
private:
    int hour_;
    int minute_;
    int second_;
};
```

Делегирующий конструктор **НЕ** может быть рекурсивным. Причем компилятор **НЕ** обязан это диагностировать (*ill-formed, no diagnostic required*). Например, прямую рекурсию `Class() :`

`Class() {}` ловят оба компилятора, а цикл из двух конструкторов, вызывающих друг друга, **GCC** молча скомпилирует в бесконечную рекурсию.

Ключевое слово `default` (*C++11*)

С помощью ключевого слова `default` можно явно попросить компилятор сгенерировать специальный метод, указав после сигнатуры `= default`. Это более выразительно, чем писать `{}` для конструктора по умолчанию. Рекомендуется к использованию.

```
class Value {
public:
    Value(int x) : x_(x) {}
    Value() = default;
    Value(const Value& other) = default;
    Value(Value&& other) = default;
    Value& operator=(const Value& other) = default;
    Value& operator=(Value&& other) = default;
    ~Value() = default;
private:
    int x_ = 0;
};
```

Ключевое слово `delete` (*C++11*)

С помощью ключевого слова `delete` можно явно попросить компилятор объявить функцию удалённой (запретить её использование), указав после сигнатуры `= delete`. Это более выразительно, чем прятать конструкторы в приватную область класса. Рекомендуется к использованию.

Можно использовать не только для конструкторов, деструкторов, но и для любых методов, операторов, шаблонных функций, функций вне классов.

```
class Value {
public:
    Value(int x) : x_(x) {}
    Value() = delete;
    Value(const Value&) = delete;
    Value& operator=(const Value&) = delete;
private:
    int x_ = 0;
};
```

Например, если класс не подразумевает сравнения на равенство или других операторов можно явно указать для них `delete`.

Удалённая функция участвует в выборе перегрузки, и если выбрана она, будет ошибка компиляции. Так можно запретить вызов с неподходящим типом:

```
void SetValue(int value);
void SetValue(double) = delete;

SetValue(20);    // OK
SetValue(20.5);  // compile error
```

Оператор преобразования

В классе возможно определить оператор преобразования, который позволяет преобразовывать пользовательский класс в другой тип.

Синтаксис: `<explicit> operator <type>() const {<body>}`

- `<explicit>` - можно запретить неявное преобразование (*C++11*)
- `<type>` - тип, к которому выполняется приведение
- возвращаемый тип не пишется, поскольку он совпадает с `<type>`

Рекомендуется помечать `const`, поскольку такой оператор не должен менять полей класса, и его можно будет вызвать для константного объекта.

Как правило, рекомендуется запрещать неявное преобразование к типу (использовать `explicit`), поскольку можно обнаружить много неожиданных мест в коде, где неявно произведено преобразование.

В том числе это касается `operator bool`. Важно помнить, что `explicit` **НЕ** помешает контекстному преобразованию к `bool` в условиях `if`, `while`, `for`, операндах `!`, `&&`, `||`, `?:`.

```
class Handle {
public:
    explicit Handle(int fd = -1) : fd_(fd) {}
    explicit operator bool() const { return fd_ >= 0; }
private:
    int fd_;
};

Handle h1{0}, h2{3};
if (!h1 && h2) {}                  // OK
// int x = h2 + 1;                 // compile error
// bool flag = h2;                 // compile error
bool flag = static_cast<bool>(h2);  // OK
```

- без `explicit` компилировалось бы `int x = h2 + 1;`: `bool` неявно превращается в `int`
- применяется `std::unique_ptr`, `std::optional`, потоках ввода-вывода

Перегрузка операторов внутри класса

Поскольку первым аргументом неявно передается указатель `this`, то перегрузка бинарных операторов внутри класса имеет один аргумент:

```
Class& operator+=(const Class& other);
Class& operator-=(const Class& other);
Class& operator*=(const Class& other);
Class& operator/=(const Class& other);
```

Операторы арифметических операций часто реализуют на основе работы присваивающих операторов:

```
Class operator+(const Class& other) const {
    Class result = *this;  // copy ctor
    result += other;       // operator +=
    return result;
}
```

- у оператора-метода левым операндом всегда является `this`, поэтому при наличии конструктора преобразования из `int` скомпилируется `obj + 5`, но **НЕ** `5 + obj`. Поэтому `+=` делают методом, а `+` - свободной функцией (см. ниже)

Операторы префиксного и постфиксного инкремента/декремента перегружаются следующим образом:

```
Class& operator++();   // ++obj
Class operator++(int); // obj++
Class& operator--();   // --obj
Class operator--(int); // obj--
```

- параметр `int` фиктивный, он нужен только чтобы отличить постфиксную форму
- постфиксный оператор возвращает копию старого значения, поэтому у возвращаемого значения нет ссылки `&`

Некоторые операторы можно перегрузить **только** как методы класса: `=`, `()`, `[]`, `->` и операторы преобразования.

Перегрузка операторов вне класса

Операторы можно перегрузить вне класса, тогда сигнатура перегружаемого оператора пишется в привычной манере, все операнды передаются явно. Но для реализации таких операторов нужен доступ к полям: через публичные методы (геттеры и сеттеры), через операторы составного присваивания внутри класса или через объявление оператора другом класса (`friend`, см. ниже).

Перегрузка инкремента и декремента вне класса:

```
Class& operator++(Class& obj);   // ++obj
Class operator++(Class& obj, int); // obj++
Class& operator--(Class& obj);   // --obj
Class operator--(Class& obj, int); // obj--
```

- объект передается по неконстантной ссылке, поскольку оператор его изменяет

Оператор побитового сдвига, отвечающий за вывод в поток `operator<<` можно перегрузить только вне класса, поскольку левый операнд - это `std::ostream`, а не наш класс.

Ключевое слово `friend`

Внутри класса с помощью ключевого слова `friend` (*friend declaration*) можно объявить дружественную функцию, класс или дружественный метод другого класса.

Сущности, объявленные дружественными, будут иметь доступ к `private` и `protected` членам класса (полям и методам).

Дружественность работает в одностороннем порядке.

Дружественность не наследуется и не транзитивна: друг моего друга — не мой друг.

```
friend void SomeFunction(int);
friend struct SomeStruct;
friend class SomeClass;
friend OtherClass;               // C++11
friend int OtherClass::Method();
```

Дружественную функцию можно сразу определить внутри класса. Такая функция **НЕ** является методом: у нее нет `this`, и она остается свободной функцией. Найти ее можно только через аргументы вызова (поиск, зависящий от аргументов, ADL), поэтому ее называют скрытым другом (*hidden friend*).

Так принято определять операторы:

```
class Money {
public:
    explicit Money(long long money) : money_(money) {}
    Money& operator+=(const Money& other) {
        money_ += other.money_;
        return *this;
    }
    friend Money operator+(const Money& lhs, const Money& rhs) {
        Money result = lhs;
        result += rhs;
        return result;
    }
```

```
    friend std::ostream& operator<<(std::ostream& os, const Money& m) {
        return os << m.money_ / 100 << '.' << m.money_ % 100 / 10 << m.money_ %
10;
    }
private:
    long long money_ = 0;
};
```

ВАЖНО: если в классе дружественная функция только объявлена, то кто угодно может написать ее определение в своем файле и получить доступ к приватным полям. Аналогично для `friend class`. Скрытый друг этого не допускает: функция уже определена, и повторное определение приведет к ошибке компиляции.

Оператор трехстороннего сравнения (*C++20*)

Начиная с *C++20* оператор `==` можно объявить `= default`: компилятор сравнит поля по порядку. Оператор `!=` писать не нужно: выражение `a != b` компилятор переписывает в `!(a == b)`.

Оператор *трёхстороннего сравнения* `<=>` (*spaceship operator*, `<compare>`) возвращает результат упорядочивания, а `<`, `<=`, `>`, `>=` компилятор выражает через него. Объявленный `= default`, он сравнивает поля лексикографически и заодно неявно объявляет `operator==` также `= default`.

```
struct Version {
    int major = 0;
    int minor = 0;
    auto operator<=>(const Version&) const = default;  // ==, !=, <, <=, >, >=
};

Version v1{1, 2}, v2{1, 10};
v1 < v2;   // true
v1 == v2;  // false
```

Такой оператор возвращает специальный объект, отвечающий за упорядочение значений:

- `std::strong_ordering` - результат сравнения `less`, `equal`, `greater`. Используется для типов, где равные значения неразличимы (целые числа)
- `std::weak_ordering` - результат сравнения `less`, `equivalent`, `greater`. Используется для типов с равными, но различимыми значениями (например, сравнение строк без учёта регистра)
- `std::partial_ordering` - результат сравнения `less`, `equivalent`, `greater`, `unordered`. Используется для типов с несравнимыми значениями (например, для значения `double` и `NaN` - все операции сравнения будут ложны).

Можно считать, что компилятор переписывает операторы сравнения следующим образом:

- `a < b` превращается в `(a <=> b) < 0`

- `a > b` превращается в `(a <=> b) > 0`
- и так далее

При `auto operator<=>(...) = default` компилятор сам выбирает самый слабый тип среди полей.

Если оператор `<=>` написан вручную, то оператор `==` **НЕ** генерируется, его нужно объявить отдельно:

```
class Money {
public:
    bool operator==(const Money&) const = default;
    std::strong_ordering operator<=>(const Money& other) const {
        return money_ <=> other.money_;
    }
private:
    long long money_ = 0;
};
```

Указатели на члены класса. Операторы `.*` и `->*`

Помимо обычных указателей, в C++ есть указатели на члены класса (*pointer to member*): на нестатическое поле или нестатический метод. Такой указатель хранит **НЕ** адрес в памяти, а то, какой член класса выбран. Объект, к которому он применяется, указывается отдельно при обращении.

Синтаксис указателя на поле:

```
<type> <class_name>::* <name> = &<class_name>::<field>;
```

- `<type>` - тип поля
- `<class_name>` - имя класса
- `<name>` - имя указателя на поле
- `<field>` - имя поля

Синтаксис указателя на метод:

```
<ret_type> (<class_name>::* <name>)(<params>) = &<class_name>::<method>;
```

- `<ret_type>` - тип возвращаемого значения функции
- `<class_name>` - имя класса
- `<name>` - имя указателя на метод
- `<params>` - типы параметров функции
- `<method>` - имя метода

Для обращения через указатель на член используются операторы:

- `.*` - для объекта (или ссылки на объект)

- `->*` - для указателя на объект

```
struct Point {
    int x = 0;
    int y = 0;
};

int Point::* coord = &Point::x;
Point p{1, 2};
Point* ptr = &p;
p.*coord = 10;     // p.x = 10
coord = &Point::y; // change ptr to another member
ptr->*coord = 20;  // p.y = 20
```

- один и тот же указатель `coord` можно применить к любому объекту `Point`

```
class Counter {
public:
    void Add(int value) { count_ += value; }
    void Sub(int value) { count_ -= value; }
    int Get() const { return count_; }
private:
    int count_ = 0;
};

void (Counter::*op)(int) = &Counter::Add;
int (Counter::*get)() const = &Counter::Get;
Counter c;
Counter* ptr_c = &c;
(c.*op)(5);      // c.Add(5)
(ptr_c->*op)(3); // c.Add(3)
(c.*get)();      // 8
```

- `&` перед именем метода обязателен, в отличие от обычных функций: `Counter::Add` без `&` - ошибка компиляции
- скобки `(c.*op)` обязательны: у вызова `()` приоритет выше, чем у `.*`, и `c.*op(5)` **НЕ** скомпилируется
- `const` входит в тип: на `const`-метод нужен указатель с `const` после параметров

Тип указателя на метод громоздкий, поэтому для него обычно заводят псевдоним:

```
using Operation = void (Counter::*)(int);

void Apply(Counter& counter, Operation op, int value) {
    (counter.*op)(value);
}

Apply(c, &Counter::Sub, 2);  // c.Sub(2)
```

Зачем это нужно? Указатель на член позволяет сделать параметром функции выбор поля или метода. Обычный параметр задает значение, а указатель на член задает, с каким членом объекта работать. Это избавляет от копирования почти одинакового кода.

Например, нужно найти сотрудника с максимальным возрастом, зарплатой или стажем. Без указателей на члены пришлось бы писать три одинаковые функции, отличающиеся только именем поля:

```
struct Employee {
    std::string name;
    int age = 0;
    int salary = 0;
    int experience = 0;
};

const Employee* MaxBy(const std::vector<Employee>& staff, int Employee::* field)
{
    const Employee* max = nullptr;
    for (const Employee& e : staff) {
        if (max == nullptr || e.*field > max->*field) {
            max = &e;
        }
    }
    return max;
}

std::vector<Employee> staff = {{"Vova", 74, 50, 20}, {"Lena", 25, 60, 30}};
MaxBy(staff, &Employee::age)->name;     // "Vova"
MaxBy(staff, &Employee::salary)->name;  // "Lena"
```

- одна функция работает с любым полем типа `int`

Аналогично указатель на метод позволяет выбирать действие во время выполнения программы. Например, сопоставить текстовые команды методам объекта без длинной цепочки `if-else`:

```
class Player {
public:
    void MoveUp() { ++pos_; }
    void MoveDown() { --pos_; }
    void Jump() { pos_ += 2; }
private:
    int pos_ = 0;
};

const std::unordered_map<std::string, void (Player::*)()> commands = {
    {"up", &Player::MoveUp},
    {"down", &Player::MoveDown},
    {"jump", &Player::Jump},
};
```

```
Player player;
(player.*commands.at("jump"))();  // player.Jump()
```

Указатель на член **НЕ** является обычным указателем: его нельзя привести к `void*` или к указателю на функцию, а размер указателя на метод бывает больше обычного указателя (на x86-64 у GCC и Clang 16 байт вместо 8). На статические поля и методы берутся обычные указатели, поскольку им не нужен объект.

Указатели на члены пишутся вручную редко, но часто передаются в библиотечный код, который сам вызывает их для нужного объекта. Например, в Qt сигналы и слоты соединяются через указатели на методы:

```
QObject::connect(button, &QPushButton::clicked, &window, &MainWindow::close);
```

- Qt сохраняет указатель на метод `&MainWindow::close` и при нажатии на кнопку `button` вызывает его для объекта `window`, то есть по сути выполняет `(window.*slot)()`
- проверка, что метод существует и подходит по параметрам, выполняется на этапе компиляции